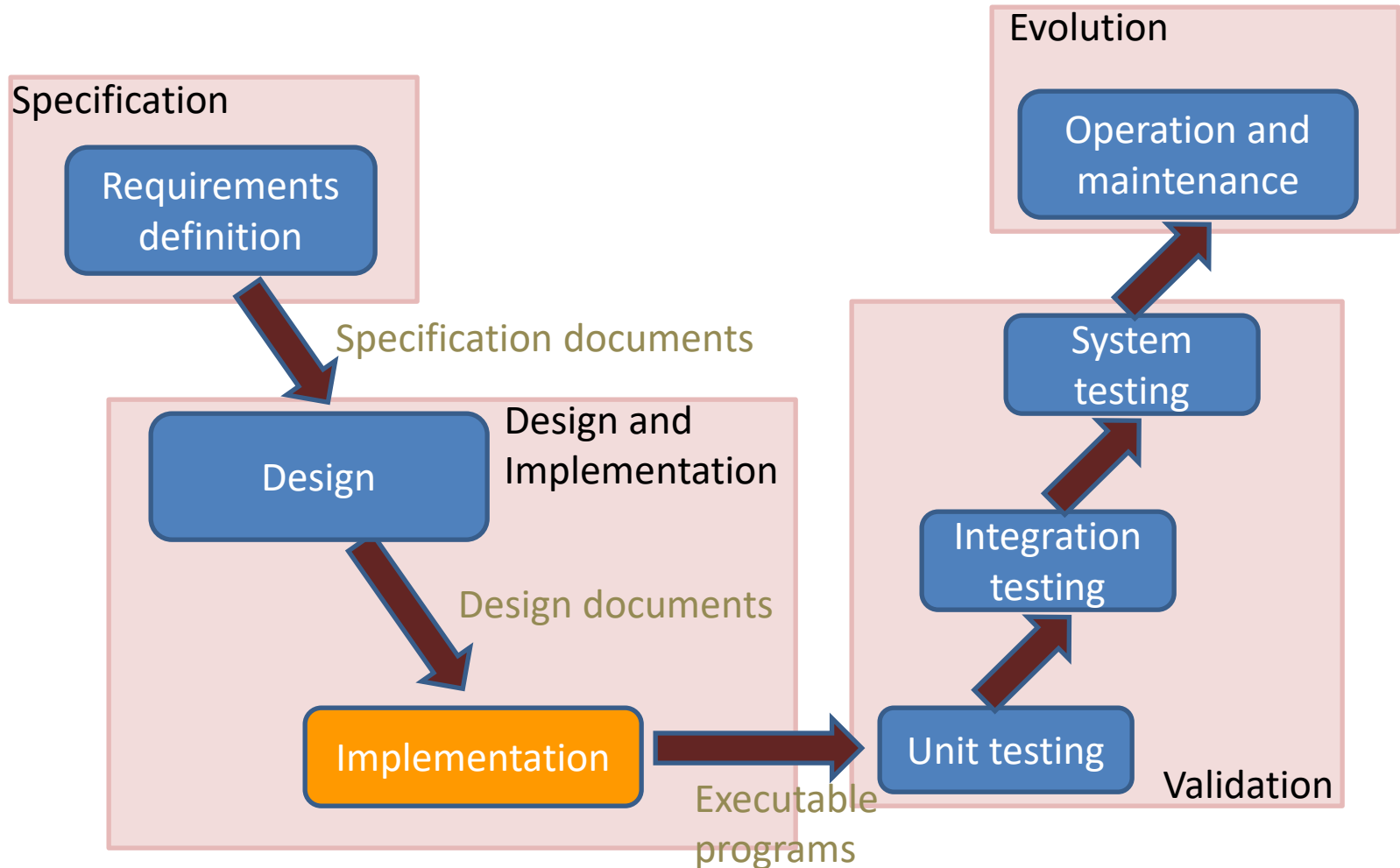


Implementation

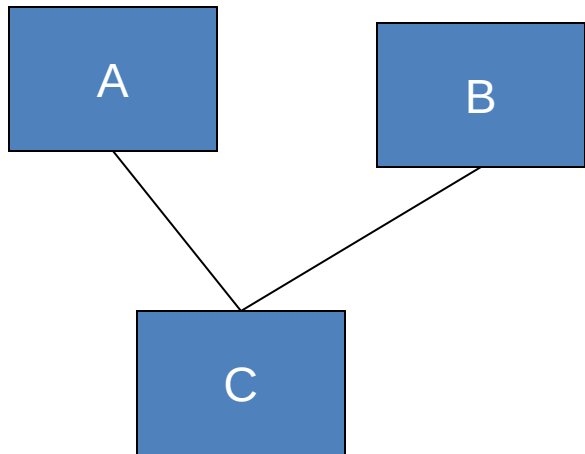
Xin Feng

Software Development Life Cycle



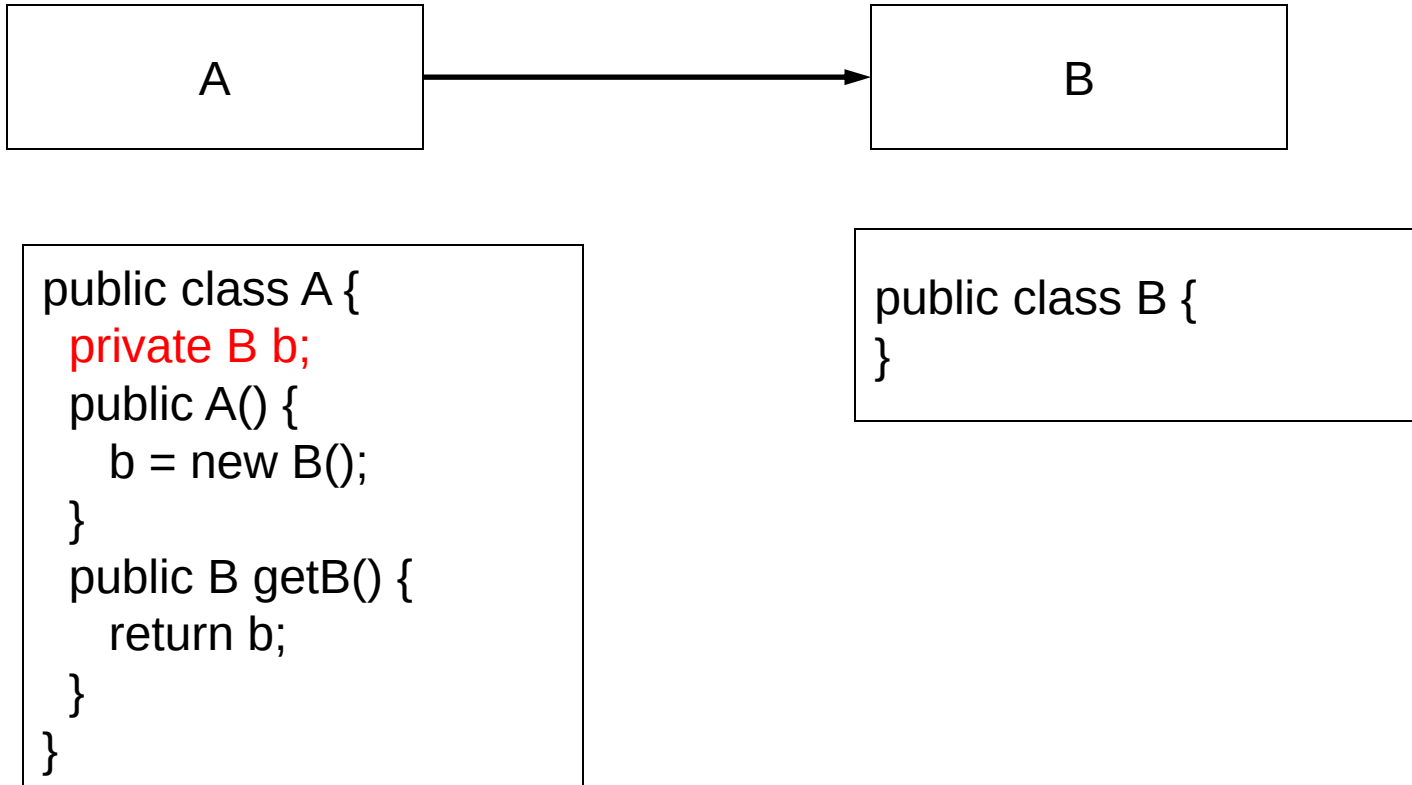
Implementation

- Mapping object diagram to source code

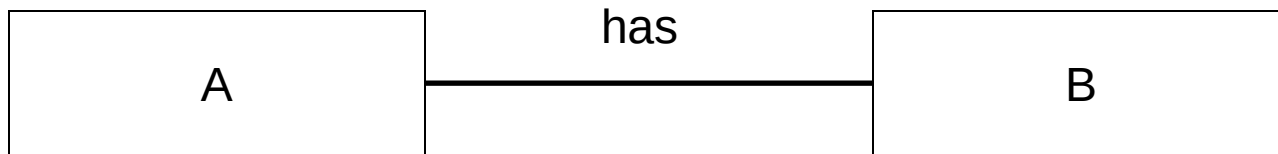


```
class A{  
}  
  
class B{  
}  
  
class C{  
}
```

One To One (unidirectional)



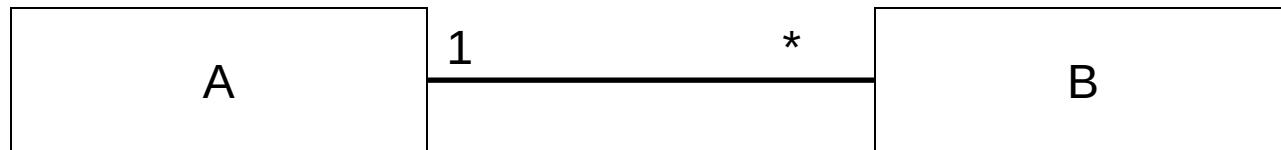
One To One (bidirectional)



```
public class A {  
    private B b;  
    public A() {  
        b = new B(this);  
    }  
    public B getB() {  
        return b;  
    }  
}
```

```
public class B {  
    private A a;  
    public B(A a) {  
        this.a = a;  
    }  
    public A getA() {  
        return a;  
    }  
}
```

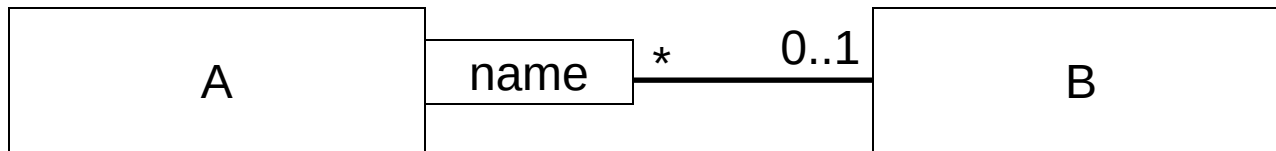
One To Many



```
public class A {  
    private Set bs;  
    public A() {  
        bs = new HashSet();  
    }  
    public void addB(B b) {  
        bs.add(b);  
        b.setA(this);  
    }  
    public void removeB(B b) {  
        bs.remove(b);  
        b.setB(null);  
    }  
}
```

```
public class B {  
    private A a;  
    public setA(A newA) {  
        if (a != newA) {  
            A oldA = a;  
            a = newA;  
            if (newA != null)  
                newA.addB(this);  
            if (oldA != null)  
                oldA.removeB(this);  
        }  
    }  
}
```

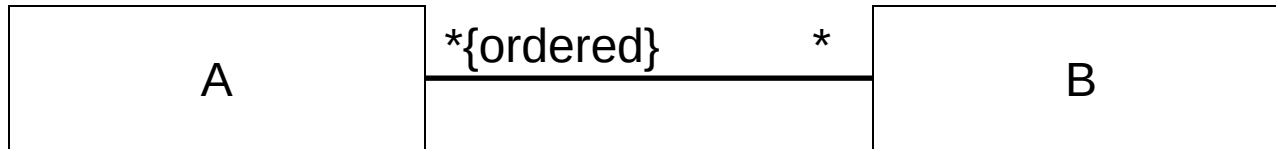
Qualifier



```
public class A {
    private Map bs;
    public void addB(String name, B b) {
        if (!bs.containsKey(name)) {
            bs.put(name, b);
            b.addA(name, this);
        }
    }
}
```

```
public class B {
    private Map as;
    public void addA(String name, A a) {
        if (!as.containsKey(a)) {
            as.put(a, name);
            a.addB(name, this);
        }
    }
}
```

Ordered



```
public class A {
    private List bs;
    public A() {
        bs = new ArrayList();
    }
    public void addB(B b) {
        if (!bs.contains(b)) {
            bs.add(b);
            b.addA(this);
        }
    }
}
```

```
public class B {
    private List as;
    public B() {
        as = new ArrayList();
    }
    public void addA(A a) {
        if (!as.contains(a)) {
            as.add(a);
            a.addB(this);
        }
    }
}
```